Ramanitharan **231901040**

Exp no:6

Implementing Fabric Channels for Privacy

Aim:

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

1. Go to the test network

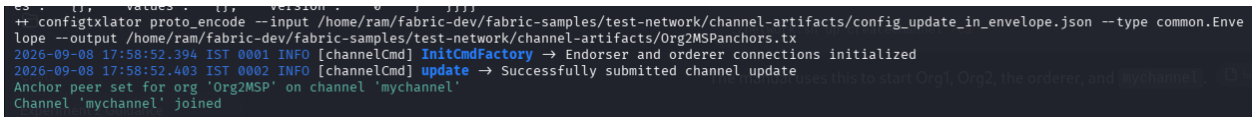
```
None
cd ~/fabric-dev/fabric-samples/test-network
```

2. Clean up Exp 5 first

```
None
./network.sh down
```

start a fresh network with CAs:

```
None
./network.sh up createChannel -ca
```



3. Set the Fabric environment

Run:

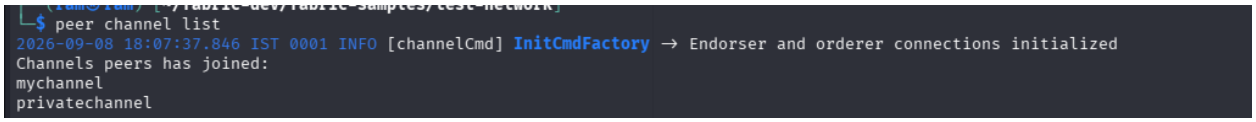
```
None
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
```

```
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizat
ions/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations
/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.c
om/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.c
om-cert.pem
```

4. Check `mychannel`

None

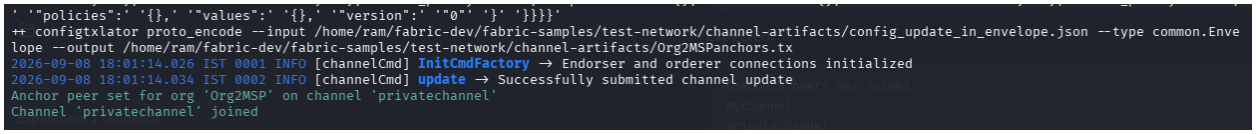
```
peer channel list
```



5. Create `privatechannel`

None

```
./network.sh createChannel -c privatechannel
```



6. Verify both channels

None

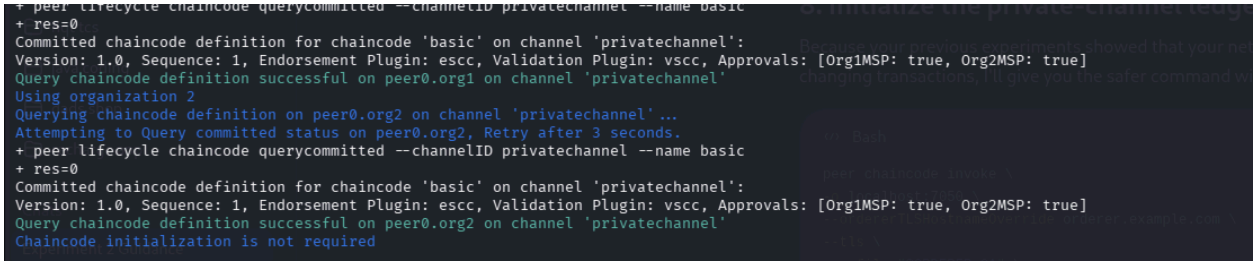
```
peer channel list
```

7. Deploy chaincode ONLY on `privatechannel`

This is the important part of the experiment.

None

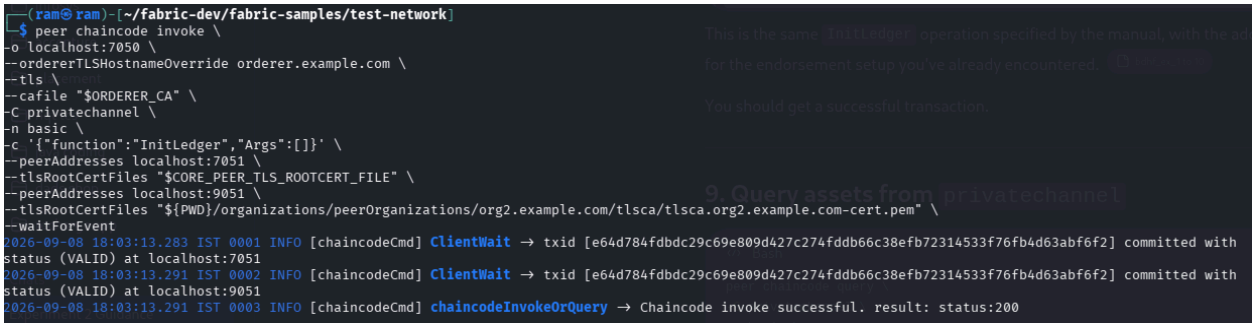
```
./network.sh deployCC \
-c privatechannel \
-ccn basic \
-ccl javascript \
-ccp ../asset-transfer-basic/chaincode-javascript
```



8. Initialize the private-channel ledger

None

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C privatechannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/tlsca
/tlsca.org2.example.com-cert.pem" \
--waitForEvent
```



9. Query assets from `privatechannel`

None

```
peer chaincode query \
-C privatechannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```

The output should look like this:

```
None
[
  {
    "AppraisedValue": 300,
    "Color": "blue",
    "ID": "asset1",
    "Owner": "Tomoko",
    "Size": 5,
    "docType": "asset"
  },
  {
    "AppraisedValue": 400,
    "Color": "red",
    "ID": "asset2",
    "Owner": "Brad",
    "Size": 5,
    "docType": "asset"
  }
]
```

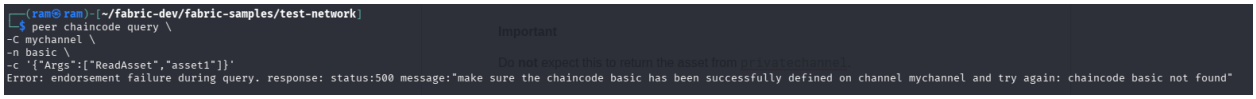
10. Demonstrate channel isolation

Now query `asset1` from **mychannel**:

```
None
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
```

Important

Do **not** expect this to return the asset from `privatechannel`.



11. Optional — prove privatechannel has the asset

Run:

```
peer chaincode query \
-C privatechannel \
-n basic \
-c '{"Args":["ReadAsset","asset1"]}'
```

You should get:

```
{
  "AppraisedValue": 300,
  "Color": "blue",
  "ID": "asset1",
  "Owner": "Tomoko",
  "Size": 5,
  "docType": "asset"
}
```

Then compare:

```
privatechannel → asset1 EXISTS
```

```
mychannel      → asset1 from privatechannel DOES NOT EXIST
```

That's the core demonstration of channel isolation.